

AI Comic Tool (星晴) 项目架构与功能梳理

生成时间：2026-01-14

1. 总览

- 单仓库三层：FastAPI 后端 + React (Vite) 前端 + Tauri 桌面壳。
- 核心目标：围绕“项目 → 剧本骨架 (集/场/镜头) → 资产 → 事件 → AI 工作流/记忆系统”提供一体化创作工作台。
- 对外项目 ID 使用 UUID (hex)，避免暴露递增整数；后端会将 uuid 映射到内部主键。

2. 目录结构 (高层)

- backend/：FastAPI + SQLAlchemy + 记忆/工作流服务层。
- react-frontend/：React 19 + TypeScript + Vite (含 src-tauri/ 桌面打包)。
- docs/：记忆系统与工作流相关设计文档 (用于理解与对齐)。
- projects/：项目占位目录 (实际运行数据通常写入 data/ 或桌面 app_data_dir)。

3. 后端架构 (FastAPI)

3.1 应用装配与运行环境

- 入口：backend/app/main.py 创建 FastAPI，注册路由并挂载 /files 静态目录。
- CORS：允许所有来源 (开发/桌面 WebView 友好)。
- 启动迁移：使用 ensure_* 轻量 SQLite 迁移函数补列/建表/建索引 (无 Alembic)。
- 数据目录：AICOMIC_DATA_DIR 优先，否则使用当前工作目录下的 data/。

3.2 主要路由模块

- /projects：项目管理 + 资产条目 (asset-items) 管理。
- /storyboard：剧本骨架 (Episode/Scene/Shot) CRUD，包含删除时的物理文件清理与视频上传。
- /assets：上传并关联资源文件 (镜头素材、资产条目素材、镜头视频等)。
- /events：事件系统 (事件 + 多粒度节点 episode/scene/shot + graph_data + matrix 聚合)。
- /ai：AI 能力聚合 (设置/模板/写作/工作流/聊天编排/上下文/运行快照)。
- /memory：记忆工程 (Evidence-first、ChangeSet 审阅、冲突、时间约束/时间块等)。

3.3 数据模型 (核心实体)

- Project：uuid(对外) + id(内部)；关联 characters(资产条目)、episodes、events。
- Episode → Scene → Shot：剧本骨架的层级结构。
- Asset：可关联资产条目 (Character) 或 Shot；存 file_path、meta_data 等。
- Event + EventNode：事件主表 + 多态节点 (target_type/target_id)。
- AiActionRun：按钮输出历史 (可用于回溯与审计)。

3.4 存储与记忆系统

- 结构化业务数据：SQLite (SQLAlchemy)。可通过 AICOMIC_DB_PATH 指定 DB 文件路径。
- 上下文与运行审计：file-first ContextStore (series_bible / project_outline / visual_dna / runs / stages)。
- 语义检索：Qdrant (本地 path 模式优先；缺依赖时降级为进程内 fallback)。
- Embedding：sentence-transformers，默认模型 BAAI/bge-m3，懒加载以降低启动开销。
- Canonical 记忆库：在 memory_store.db 中维护 evidences/entities/events/snapshots/changesets/conflicts 等表簇。

4. 前端架构 (React + Vite)

4.1 页面与导航

- /script：剧本主工作台 (编辑集/场/镜头 + AI Chat/Workflow)。
- /events、/events/flow：事件矩阵与事件流视图。
- /assets：资产库。
- /context：上下文查看/编辑。
- /runs：运行快照与阶段审计。
- /settings/ai、/settings/prompts：AI 设置与 Prompt 模板管理。

4.2 API 调用与项目选择

- 统一 axios client：支持 VITE_API_BASE_URL 与桌面端注入 window.__AICOMIC_API_BASE_URL__。
- projectId 存在 localStorage (key=aicomic.projectId)，并在启动时自动选择首个项目。
- 对外 projectId 为 uuid 字符串；后端会映射到内部整数主键。

5. 桌面端 (Tauri) 集成方式

- 启动时拉起 Python + uvicorn (固定端口 8000)。
- 向后端注入 AICOMIC_DB_PATH 与 AICOMIC_DATA_DIR，确保桌面端数据隔离在 app_data_dir。
- 向前端注入 API baseURL (http://127.0.0.1:8000)，前端监听事件自动切换。

6. 核心数据流 (端到端)

- 项目 → 剧本：/projects 选择项目 → /storyboard 拉取与编辑 Episode/Scene/Shot。
- 剧本 → 资产：上传素材/视频并写入 Asset/Shot.video_path，前端通过 /files 访问。
- 剧本 → AI：调用 /ai/* 进行写作/工作流；过程可写入 runs/stages 供前端轮询展示步骤。
- 原文 → 记忆：/memory/evidence/ingest 切片 → /memory/extract/changeset LLM 抽取 → ChangeSet 审阅 → 写入 canonical 真值库。

7. 功能清单 (当前已具备)

- 项目管理：创建/改名/删除；删除会清理数据与文件目录。
- 剧本骨架：集/场/镜头 CRUD；镜头视频上传；删除时清理关联文件。
- 资产条目：按 category 分类的人设/背景等条目管理与素材上传。

- 事件系统：事件维护、节点绑定到 episode/scene/shot、图数据保存与矩阵聚合接口。
- AI：设置与测试、Prompt 模板、写作能力（大纲/剧本）、工作流（script/storyboard）、聊天编排与多步执行。
- 运行审计：按钮输出历史 + runs-files/stages 文件快照回放。
- 记忆工程：Evidence-first、ChangeSet、Conflict、时间约束/时间块、canonical 实体/事件/快照/状态变更存储。

8. 可扩展点（建议落点）

- 新增 AI 能力：新增 backend/app/routers/ai_*.py 子模块并在 ai.py 聚合；前端在 api/client.ts 增加封装与页面入口。
- 新增工作流：在 ai_workflows.py 增加 workflow + apply，并复用 runs-files/stages 做可观测性。
- 增强记忆系统：扩展 memory_store 的 canonical 表与检索策略；向量侧可切换远端 Qdrant。
- 资产体系演进：继续围绕 asset-items(category) 扩展资产类型（视觉/设定/道具/地点等）。

输出文件：/Users/linjiacheng/Documents/GitHub/aicomc-tool/aicomc-tool-architecture.pdf